🗄️ EduTrack — Database Document

Academic Management Platform

VTU Internship 2026 — Team 15

1. Creating the Database

The EduTrack database was created using MySQL Workbench.

Steps:

1. Open MySQL Workbench
2. Connect to localhost with root credentials
3. Run the following SQL statement:

```
CREATE DATABASE edutrack;
USE edutrack;
```

The database name `edutrack` is referenced in `application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost:3306/edutrack
```

2. Creating Tables

EduTrack has **11 tables** in total. The tables were created in two ways:

2a. Using Schema.sql

A `Schema.sql` file is provided in the `EduTrack-Database/` folder. It contains all `CREATE TABLE` statements and can be imported directly into MySQL Workbench:

- File → Open SQL Script → select `Schema.sql` → click the lightning bolt ⚡

2b. Using Hibernate DDL Auto

Spring Boot with Hibernate is configured to automatically create and update tables on startup:

```
spring.jpa.hibernate.ddl-auto=update
```

This means when the backend starts, Hibernate reads all `@Entity` classes and automatically creates the corresponding tables if they don't exist, or adds missing columns if the table already exists.

Tables Created

TABLE	DESCRIPTION
admins	Administrator accounts
teachers	Teacher accounts and profiles
students	Student accounts and profiles
courses	Courses created by teachers
attendance	Daily attendance records
assignments	Assignments per course
submissions	Student assignment submissions
marks	Student marks and grades
course_materials	Study materials uploaded by teachers
otp_verification	OTP records for password reset
system_settings	Platform-wide configuration

3. Relations and ER Diagram

3a. Table Relationships

The tables are related through foreign keys:

RELATIONSHIP	TYPE	FOREIGN KEY
teachers → courses	One to Many	courses.teacher_id
courses → attendance	One to Many	attendance.course_id
students → attendance	One to Many	attendance.student_id
courses → assignments	One to Many	assignments.course_id

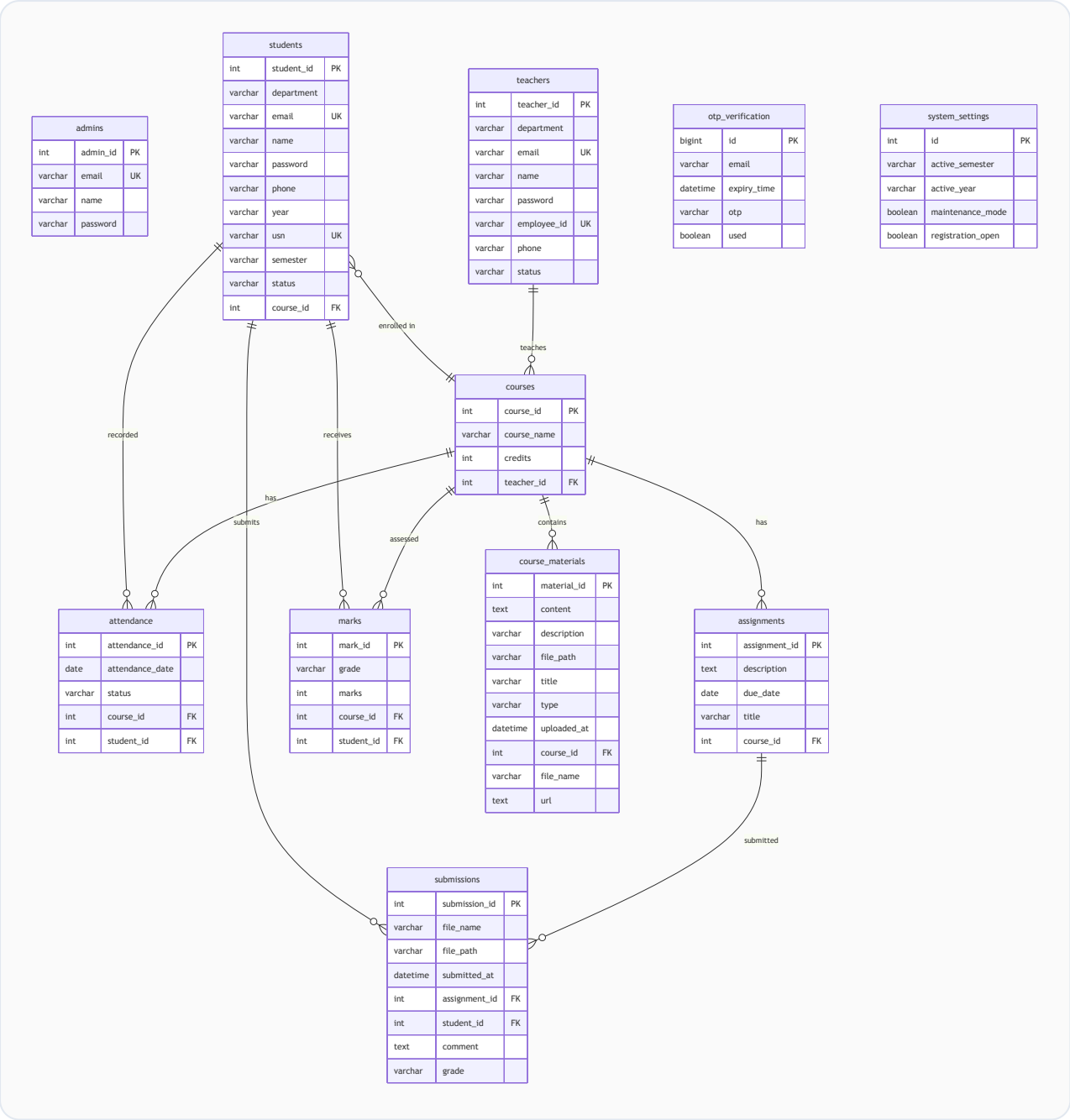
RELATIONSHIP	TYPE	FOREIGN KEY
assignments → submissions	One to Many	submissions.assignment_id
students → submissions	One to Many	submissions.student_id
courses → marks	One to Many	marks.course_id
students → marks	One to Many	marks.student_id
courses → course_materials	One to Many	course_materials.course_id
students → courses	Many to One	students.course_id

Standalone tables (no foreign keys):

- `otp_verification` — looked up by email string, serves all three roles
- `system_settings` — single row configuration table
- `admins` — no relationships to other tables

3b. ER Diagram

The ER diagram below was generated using **MySQL Workbench Reverse Engineer** and recreated for documentation.



3c. Using ER Diagram with AI

The ER diagram was used as input to AI tools to generate and verify SQL statements. The diagram clearly shows all table structures, column types, primary keys, foreign keys, and relationships — making it easy to generate accurate `CREATE TABLE` statements and validate the schema design.

4. OTP Expiration

4a. How OTP Expiry Works

When an OTP is generated, an expiry timestamp is stored in the `otp_verification` table:

```
// OtpService.java
otpEntity.setExpiryTime(LocalDateTime.now().plusMinutes(10));
```

The OTP expires **10 minutes** after generation.

4b. Expiry Validation

When a user submits an OTP, the backend checks if it has expired:

```
// OtpService.java
if (otpEntity.getExpiryTime().isBefore(LocalDateTime.now())) {
    System.out.println("[OTP] OTP expired for: " + email);
    return false;
}
```

4c. OTP Cleanup

Old OTPs are **automatically deleted from the table** before a new OTP is generated for the same email:

```
// OtpService.java
otpRepo.deleteByEmail(email);  // Delete old OTP first
// Then create and save new OTP
```

This ensures only one active OTP exists per email at any time.

4d. Single-Use OTP

After successful verification, the OTP is marked as used so it cannot be reused:

```
otpEntity.setUsed(true);
otpRepo.save(otpEntity);
```

The validation checks this flag:

```
if (otpEntity.isUsed()) {
    return false;  // Already used
}
```

4e. OTP Table Structure

```sql
CREATE TABLE otp_verification (
    id          BIGINT   NOT NULL AUTO_INCREMENT,
    email       VARCHAR(100) NOT NULL,
    otp         VARCHAR(6)   NOT NULL,
    expiry_time DATETIME     NOT NULL,
    used        BOOLEAN      NOT NULL DEFAULT FALSE,
    PRIMARY KEY (id)
);
```

5. Password Encryption (BCrypt)

All passwords in EduTrack are encrypted using **BCrypt** before being stored in the database.

5a. What is BCrypt?

BCrypt is a password hashing algorithm that:

- Automatically generates a random **salt** for each password
- Produces a different hash every time even for the same password
- Is computationally expensive — making brute force attacks slow
- Is the industry standard for password storage

5b. Implementation

```java
// AuthService.java
private final BCryptPasswordEncoder encoder = new
BCryptPasswordEncoder();

// Encoding password before saving
student.setPassword(encoder.encode(rawPassword));

// Verifying password during login
boolean matches = encoder.matches(rawPassword, storedHash);
```

5c. Password Never Returned in API

The `password` field is annotated with `@JsonProperty(access = WRITE_ONLY)` on all entity classes, ensuring the hashed password is never included in API responses:

```java
// Student.java
@JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
@Column(nullable = false)
private String password;
```

5d. BCrypt Hash Example

A raw password like `Admin@123` is stored as a hash like:

```
$2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lHHi
```

The hash cannot be reversed — the only way to verify is using `encoder.matches()`.

6. Cascade Delete Rules

When a record is deleted, related records are automatically deleted to maintain data integrity:

DELETE ACTION	CASCADES TO
Delete teacher	All their courses, attendance, marks, assignments, submissions, materials
Delete student	All their attendance, marks, submissions
Delete course	All attendance, marks, assignments, submissions, materials for that course
Delete assignment	All submissions for that assignment

This is handled both at the JPA level (`@OneToMany(cascade = CascadeType.ALL)`) and manually in the controller before deletion.

7. Database Configuration

```
# MySQL Connection
spring.datasource.url=jdbc:mysql://localhost:3306/edutrack?
useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true
spring.datasource.username=root
spring.datasource.password=YOUR_PASSWORD
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Hibernate
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```